

Tremvim - Maintenance Guide

1. Introduction

Tremvim is a rideshare platform for university students, allowing drivers and passengers to coordinate shared rides. It supports user registration with student and driver verification, real-time ride matching and chat, push notifications, GPS autocomplete for Israeli cities, and ride reviews.

This guide helps future maintainers understand the structure of the project and how to safely extend and maintain it.

2. Project Structure Overview

Root Directory

None	
tremvim-main/	
├─ core/	← Django project settings, main API, Celery, ASGI
├─ users/	← User accounts, auth, student/driver verification
├─ rides/	← Ride creation, matching, booking, lifecycle
├─ notifications/	← Push notifications, WebSocket consumers
├─ gps/	← GPS autocomplete for Israeli cities/streets
├─ reviews/	← Ride ratings and reviews
├─ safety/	← Safety features (emergency contacts)
├─ tests/	← Shared test factories and utilities
├─ tremvim-frontend/	← Expo / React Native mobile app
├─ nginx/	← Nginx reverse proxy config
├─ scripts/	← Utility scripts (local DB setup)
├─ media/, private_media/	← User-uploaded files
├─ manage.py	← Django CLI entry point
├─ requirements.txt	← Python dependencies
├─ Dockerfile	← Backend container definition
├─ docker-compose.yml	← Multi-service orchestration
├─ deploy.bat	← Windows deployment script

- └─ DEPLOY_SETUP.md
- └─ pyproject.toml
- ← SSH key setup guide
- ← Pytest + coverage configuration

Backend Django Apps

None

core/

- └─ settings.py
- Celery, CORS)
- └─ api.py
- aggregation
- └─ asgi.py
- routing
- └─ celery.py
- └─ middleware.py
- └─ throttle.py
- ← All Django configuration (MySQL, Redis,
- ← NinjaAPI root, JWT auth, router
- ← ASGI entry point (Daphne), WebSocket
- ← Celery configuration
- ← Health check, JWT auth for WebSockets
- ← Custom rate limiting

users/

- └─ models.py
- DriverStatus
- └─ api.py
- └─ services.py
- └─ tasks.py
- └─ schemas.py
- └─ license_ocr.py
- └─ migrations/
- └─ tests/
- ← CustomUser, StudentVerificationStatus,
- ← Auth endpoints, profile, verification
- ← Business logic for user operations
- ← Celery tasks (email, uploads)
- ← Pydantic request/response schemas
- ← Google Cloud Vision for license OCR

rides/

- └─ models.py
- └─ api.py
- manage)
- └─ services.py
- └─ tasks.py
- └─ consumers.py
- └─ signals.py
- └─ migrations/
- └─ tests/
- ← Ride, RideLocation, RideStatus
- ← Ride endpoints (create, search, join,
- ← Ride matching, validation, status updates
- ← Celery tasks (reminders, status updates)
- ← WebSocket consumers for ride chat
- ← Django signals for ride events

```
notifications/
├─ models.py           ← ExpoPushToken storage
├─ api.py              ← Device token registration
├─ services.py         ← Push notification sending
├─ push_service.py     ← Low-level Expo push server interface
├─ tasks.py            ← Async notification Celery tasks
├─ consumers.py        ← WebSocket real-time consumers
├─ migrations/
└─ tests/

gps/
├─ services.py         ← City/street autocomplete logic
├─ api.py              ← Autocomplete API endpoint
├─ autocomplete_data.csv ← 9.5 MB dataset of Israeli cities/streets
└─ tests/
```

Frontend Structure (trem-pim-frontend/)

```
None
trem-pim-frontend/
├─ app/                ← File-based routes (Expo Router)
│   ├─ (auth)/         ← Login, Register screens
│   ├─ (tabs)/         ← Main tabbed interface
│   ├─ driver/         ← Driver-specific screens
│   ├─ rides/          ← Ride browsing and management
│   └─ _layout.tsx     ← Root layout
├─ src/
│   ├─ api/            ← Axios API client and React Query hooks
│   ├─ components/     ← Reusable UI components
│   ├─ contexts/       ← React Context (auth, rides)
│   ├─ hooks/          ← Custom hooks
│   ├─ i18n/           ← Internationalization (Hebrew support)
│   └─ types/          ← TypeScript type definitions
├─ assets/             ← Images, fonts
├─ app.json            ← Expo app configuration
├─ eas.json            ← Expo Application Services build config
├─ tailwind.config.js  ← NativeWind (Tailwind CSS) configuration
└─ tsconfig.json       ← TypeScript configuration
```

└─ package.json

└─ Frontend dependencies and scripts

3. Tech Stack

Component	Technology
Backend Framework	Django 4.2 + Django Ninja
API Authentication	JWT (ninja-jwt) with custom suspension checks
Real-Time Layer	Django Channels + Redis (WebSockets)
Task Queue	Celery with Redis broker + Beat scheduler
Database	MySQL 8.0 (utf8mb4)
Cache & Pub/Sub	Redis 7
Mobile Frontend	Expo (React Native 0.81) + Expo Router
Mobile UI	NativeWind (Tailwind CSS for React Native)
State Management	TanStack React Query 5 + Axios
ASGI Server	Daphne
Reverse Proxy	Nginx 1.25 (Alpine) with SSL/TLS
Containerization	Docker + Docker Compose
OCR	Google Cloud Vision API (license verification)
Push Notifications	Expo Server SDK
Location Services	OpenRouteService API
Testing	Pytest, pytest-django, pytest-cov, factory-boy
Language	Python 3.11 (backend), TypeScript (frontend)

4. Environment Setup

Requirements:

- Backend: Python 3.11+
- Frontend: Node.js 18+ with npm
- Database: MySQL 8.0 (local) or Docker
- Cache/Broker: Redis 7 (local or Docker)
- Mobile: Expo Go app (iOS/Android) for local testing
- Backend Setup (Local Development)

Option A – With Docker (Recommended for Redis only)

Shell

```
# Start only Redis locally (no full production stack)
docker compose up redis -d
```

```
# Then run Django manually:
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt
```

```
# Set up local MySQL database (run once):
scripts\db\setup_local_db.bat
```

```
# Copy .env and fill in your values:
copy .env.example .env
```

```
# Apply migrations and start the server:
python manage.py migrate
python manage.py runserver
```

Option B – Full Docker Production Stack

Shell

```
# Requires VPN connection to server (for production deploy only)
docker compose --profile prod up -d --build
```

Frontend Setup

Shell

```
cd trempim-frontend
npm install
npm start          # Start Expo dev server (scan QR code with Expo Go)
npm run android    # Run on Android emulator
npm run ios        # Run on iOS simulator
npm run web        # Run web version in browser
```

The frontend connects to the API using the `API_URL` in `trempio-frontend/.env`:

```
None
API_URL=http://132.73.84.64/api      # Production server
# or
API_URL=http://<your-local-ip>:8000/api  # Local development
```

Required Environment Variables (.env)

Create a `.env` file at the project root based on the existing `.env`. Key variables:

Variable	Description
DJANGO_SECRET_KEY	Django secret key
DJANGO_DEBUG	True for development
MYSQL_DATABASE	Database name (trempio)
MYSQL_USER	Database user
MYSQL_PASSWORD	Database password
MYSQL_HOST	Database host (127.0.0.1)
REDIS_URL	Redis connection string
EMAIL_HOST_USER	Gmail address for sending emails
EMAIL_HOST_PASSWORD	Gmail app password
GOOGLE_APPLICATION_CREDENTIALS	Path to Google Cloud Vision JSON key
OPENROUTESERVICE_API_KEY	API key for route calculations
DEPLOY_USER	Your server SSH username
DEPLOY_HOST	Server IP (132.73.84.64)

5. Adding New Features / Updating Current Features

Full-Stack Feature Flow

Follow this order when adding a new feature end-to-end:

1. Frontend (trempio-frontend/)

- Locate or create the relevant screen in `app/` (Expo Router file-based routing).
- Add API calls in `src/api/` using `axios` and `React Query` hooks.
- Add reusable UI components in `src/components/`.

- Example: a new "Cancel Ride" button calls DELETE /api/rides/{id}/.

2. API Communication (Backend)

- Add or update the route in the relevant api.py file (e.g., rides/api.py).
- Use @router.delete(...) or the appropriate HTTP method decorator.
- Protect with auth=AuthBearer() (custom JWT auth class).
- Define request/response schemas in schemas.py using Pydantic.

3. Service Layer

- Implement the business logic in the app's services.py.
- Keep API handlers thin — they should only validate input and call a service method.
- Catch and raise appropriate HTTP exceptions with clear error messages.

4. Business Logic & Models

- If new database columns or tables are needed, update models.py.
- Run python manage.py makemigrations <app> and commit the migration file.
- Apply with python manage.py migrate.

5. Async Tasks (Celery)

- For heavy operations (emails, push notifications, scheduled jobs), add a task to the app's tasks.py.
- Call it with .delay() or .apply_async() from the service layer.
- For scheduled/periodic tasks, register it in core/celery.py using Celery Beat.

6. Real-Time Updates (WebSockets)

- Add a consumer in the app's consumers.py (inherits from AsyncJsonWebsocketConsumer).
- Register the URL pattern in routing.py.
- Wire it into core/asgi.py under the URLRouter.

7. Testing

- Add unit tests in the app's tests/ directory.

Use factories from tests/factories.py to create test data.

- Run tests locally: pytest
- Minimum coverage requirement: 70%

8. Deployment

- Push changes to prod branch on GitHub.
- CI runs automatically (see Section 7).
- Deploy to server using deploy.bat (see Section 6).

6. Connecting to the Server and Deploying

Server: 132.73.84.64 (VPN connection required - connect to BGU VPN first)

GitHub Repository: <https://github.com/DanielShamirWork/trempim-main>

One-Time SSH Key Setup (Windows)

Do this once so deploy.bat runs without password prompts:

Step 1 - Generate your SSH key:

None

```
ssh-keygen -t ed25519 -f "$env:USERPROFILE\.ssh\id_trempim" -N ''
```

Step 2 - Add your public key to the server (you will be prompted for your password once):

None

```
$key = Get-Content "$env:USERPROFILE\.ssh\id_trempim.pub"
ssh user@132.73.84.64 "mkdir -p ~/.ssh && echo '$key' >>
~/.ssh/authorized_keys && chmod 700 ~/.ssh && chmod 600
~/.ssh/authorized_keys"
```

Replace user with your server username.

Step 3 - Set your credentials in `.env`:

None

```
DEPLOY_USER=your_username
DEPLOY_HOST=132.73.84.64
```


Deploying

None

```
.\deploy.bat
```

The script does the following automatically:

1. Connects to the server via SSH
2. Pulls the latest code from the prod branch
3. Saves pre-deploy Docker logs to ~/logs/
4. Runs docker compose --profile prod down
5. Runs docker compose --profile prod up -d --build (rebuilds all containers, runs migrations, collects static files)
6. Saves startup logs and prunes old Docker images

Manual File Update (Alternative)

To update a single file without a full redeploy, use `scp`:

None

```
# Windows PowerShell
```

```
scp C:\path\to\local\file.py  
user@132.73.84.64:/home/user/trempim-main/path/on/server/
```

For example:

None

```
scp rides\services.py  
daniel@132.73.84.64:/home/daniel/trempim-main/rides/services.py
```

Direct Server Access via VSCode

1. Open VSCode.
2. Press Ctrl+Shift+P → select Remote-SSH: Add New SSH Host.
3. Enter: ssh your_username@132.73.84.64
4. Choose ~/.ssh/config as the config file.

5. Reopen Command Palette → Remote-SSH: Connect to Host → select the host.

7. CI/CD Pipeline

The project uses GitHub Actions (`.github/workflows/django.yaml`).

Trigger: Push or pull request to prod or dev2 branches.

What it does:

1. Spins up Redis and MySQL 8.0 as services.
2. Installs Python 3.12 and all dependencies from `requirements.txt`.
3. Grants MySQL test database privileges.
4. Runs `pytest` with coverage reporting.

Required GitHub Secrets (set in repo Settings → Secrets):

Secret	Description
EMAIL_HOST_USER	Gmail address for SMTP
EMAIL_HOST_PASSWORD	<persisted-output>

CI-specific behaviour: Celery runs in eager mode (no worker needed), real emails are disabled, and a test Django secret key is used.

8. Docker Services

The production stack is defined in `docker-compose.yml` and launched with `--profile prod`:

Service	Image	Role
redis	redis:7	Message broker for Celery and Django Channels
mysql	mysql:8.0	Primary database
web	(built from Dockerfile)	Django/Daphne ASGI server on port 8000

celery	(built from Dockerfile)	Background task worker + Beat scheduler
nginx	nginx:alpine	Reverse proxy, SSL termination on ports 80/443

The **web** container startup sequence:

None

```
collectstatic → migrate → daphne -b 0.0.0.0 -p 8000  
core.asgi:application
```

Important: Never delete the data/ volume directory on the server - it contains the persistent MySQL database files.

9. Running Tests

Shell

```
# Run all tests
```

```
pytest
```

```
# Run with verbose output
```

```
pytest -v
```

```
# Run a specific app's tests
```

```
pytest users/tests/
```

```
# Run with coverage report
```

```
pytest --cov
```

```
# Run tests in parallel
```

```
pytest -n auto
```

Coverage minimum is 70%. The CI will fail if this threshold is not met.

Test data is generated using Factory Boy factories defined in tests/factories.py. Use these in your own tests rather than creating model instances manually.

10. Project Architecture Notes

Service Layer Pattern: Business logic lives in services.py files within each app. API handlers in api.py are kept thin — they validate input and delegate to services.

Dependency Injection: The project uses anydi for loose coupling between services. Service dependencies are declared in dep.py files.

JWT Authentication: A custom SecureJWTAuth class (in core/api.py) extends the standard JWT auth to also check whether a user is suspended or unverified before allowing access.

WebSockets: Django Channels handles WebSocket connections. Two consumer groups exist: ride chat (rides/consumers.py) and real-time notifications (notifications/consumers.py). Both require an authenticated user.

Push Notifications: Expo Push Token is stored per user device. The notifications/push_service.py module sends notifications through the Expo push gateway.

Migrations: Always commit migration files. Never squash or delete migrations from the history. If a schema change is needed on the live server, SSH in and run python manage.py migrate inside the web container:

```
Shell
ssh user@132.73.84.64
cd ~/trem-pim-main
docker exec -it trem-pim-main-web-1 python manage.py migrate
```

11. Contact Info

Developer	Email
Daniel Shamir	shamirda@post.bgu.ac.il
Avikam Levi	avikaml@post.bgu.ac.il
Eitan Dubinski	eitandu@post.bgu.ac.il
Tal Ginzburg	talgin@post.bgu.ac.il